ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»

Параллельные вычисления

ЛАБОРАТОРНАЯ РАБОТА № 7

Тема: SIMD-оптимизация сверточных операторов обработки изображений на примере оператора Собела с использованием различных наборов команд

Группа: БПМ-22-ПО-1

Студент: Нургалиев Родион Дамирович

Номер в группе: 20

Вариант: 10

Ссылка на репозиторий с кодом:

https://github.com/LLrodyaLL/parallel_computing_misis_nurgaliev_r_d_bpm_22_po_1/tree/main/lab07

2026

1. Цель работы

Познакомиться с принципами организации и программирования векторных расширений системы команд процессоров семейства x86. Реализовать SIMD-оптимизацию оператора Шарра с использованием набора команд AVX2, убедиться в корректности результатов путём сравнения со скалярной реализацией и оценить достигнутый выигрыш в скорости обработки.

2. Описание условия решаемой задачи (Вариант 10)

Оператор Шарра применяется для выделения границ на полутоновых изображениях. Он является улучшенной версией оператора Собела с более высокой изотропностью отклика градиента. Математически полутоновое изображение представляется матрицей, элементы которой - целые числа из диапазона [0; 255].

Для каждого внутреннего пикселя (x, y) производится свёртка 3×3-окрестности с двумя масками:

Горизонтальная маска Gx:		
-3	0	+3
-10	0	+10
-3	0	+3

Вертикальная маска Gy:		
-3	-10	-3
0	0	0
+3	+10	+3

Итоговое значение пикселя вычисляется по формуле:

$$d(x,y) = \mathrm{clamp}(\ \mathrm{sqrt}(Gx^2 + Gy^2) \times norm,\ \ 0,\ \ 255\ )$$

где нормировочный коэффициент $norm = 1 / (16 \times \sqrt{2}) \approx 0.04419$ обеспечивает попадание результата в диапазон [0; 255]. Краевые пиксели обнуляются.

3. Схема преобразования значений

3.1. Скалярная реализация

Для каждого внутреннего пикселя извлекаются восемь соседних значений из строк prev, curr и next, затем последовательно вычисляются градиенты и итоговое значение:

Шаг	Операция
1	Загрузка 8 соседних пикселей: tl, tc, tr, ml, mr, bl, bc, br
2	Gx = 3*(tl−tr) + 10*(ml−mr) + 3*(bl−br)
3	Gy = 3*(tl−bl) + 10*(tc−bc) + 3*(tr−br)
4	mag = sqrt(Gx^2 + Gy^2) × norm
5	dst[x,y] = clamp(mag, 0, 255) → запись в uint8

3.2. SIMD-реализация (AVX2)

AVX2-версия обрабатывает 8 пикселей одновременно с помощью 256-битных YMM-регистров (8 × float32). Схема обработки одной порции из 8 пикселей:

Шаг	Операция	Инструкция AVX2
1	Загрузка 8 байт из памяти	_mm_loadl_epi64
2	Расширение uint8 → int32	_mm256_cvtepu8_epi32
3	Конвертация int32 → float32	_mm256_cvtepi32_ps
4	Gx = 3*(tl−tr)+10*(ml−mr)+3*(bl−br)	_mm256_mul_ps / sub / add
5	Gy = 3*(tl−bl)+10*(tc−bc)+3*(tr−br)	_mm256_mul_ps / sub / add
6	mag = sqrt(Gx^2+Gy^2) × norm	_mm256_sqrt_ps / _mm256_mul_ps
7	Clamp [0;255], float→int, запись 8 байт	_mm256_cvttps_epi32 / store

Хвост строки (менее 8 пикселей у правого края) обрабатывается скалярным fallback-кодом.

4. Листинг программы (ключевые фрагменты)

4.1. Скалярная реализация (scharr_scalar.cpp)

Основной цикл скалярной обработки:

```
for (size_t y = 1; y + 1 < src.height; ++y) {
  for (size_t x = 1; x + 1 < src.width; ++x) {
    int tl=src.at(x-1,y-1), tc=src.at(x,y-1), tr=src.at(x+1,y-1);
    int ml=src.at(x-1,y),                     mr=src.at(x+1,y);
    int bl=src.at(x-1,y+1), bc=src.at(x,y+1), br=src.at(x+1,y+1);

    float gx = 3*(tl-tr) + 10*(ml-mr) + 3*(bl-br);
    float gy = 3*(tl-bl) + 10*(tc-bc) + 3*(tr-br);
    float mag = std::sqrt(gx*gx + gy*gy) * kScharrNorm;
    dst.at(x,y) = saturate_to_byte(mag);
  }
}
```

4.2. Вспомогательные функции AVX2 (scharr_simd_avx2.cpp)

Загрузка 8 пикселей из памяти и конвертация в float32:

```
inline __m256 load_u8x8_as_ps(const uint8_t* ptr) {
  __m128i bytes = _mm_loadl_epi64((__m128i*)ptr);
  __m256i ints  = _mm256_cvtepu8_epi32(bytes);
  return _mm256_cvtepi32_ps(ints);
}
```

Запись 8 результатов в память с насыщением:

```
inline void store_truncated_u8x8(__m256 values, uint8_t* dst) {
  __m256 clamped = _mm256_min_ps(_mm256_max_ps(values,
                   _mm256_setzero_ps()), _mm256_set1_ps(255.f));
  __m256i ints = _mm256_cvttps_epi32(clamped);
  alignas(32) int tmp[8];
  _mm256_store_si256((__m256i*)tmp, ints);
  for (int i=0; i<8; ++i) dst[i] = (uint8_t)tmp[i];
}
```

4.3. Основной SIMD-цикл (scharr_simd_avx2.cpp)

```
const __m256 c3=_mm256_set1_ps(3.f), c10=_mm256_set1_ps(10.f);
const __m256 norm=_mm256_set1_ps(kScharrNorm);
```

```
for (size_t x=1; x+7 < src.width-1; x+=8) {
  __m256 tl=load(prev+x-1), tc=load(prev+x), tr=load(prev+x+1);
  __m256 ml=load(curr+x-1),                  mr=load(curr+x+1);
  __m256 bl=load(next+x-1), bc=load(next+x), br=load(next+x+1);

  __m256 gx = c3*(tl-tr) + c10*(ml-mr) + c3*(bl-br);
  __m256 gy = c3*(tl-bl) + c10*(tc-bc) + c3*(tr-br);
  __m256 mag = _mm256_sqrt_ps(gx*gx + gy*gy) * norm;
  store_truncated_u8x8(mag, out+x);
}
for (; x+1 < src.width; ++x)   // скалярный fallback
  out[x] = scalar_one_pixel(src, x, y);
```

5. Результаты сопоставления версий

После запуска программы на тестовом изображении 1920×1080 выполнялось попиксельное сравнение результатов скалярной и SIMD-реализаций. Результаты приведены в таблице ниже.

Параметр	Значение
Размер входного изображения	1920 × 1080 пикселей
Количество пикселей с расхождением	0
Максимальное абсолютное отличие	0
Результат сопоставления	Полное совпадение (100%)

Расхождений между скалярным и SIMD-вариантами не обнаружено. Оба выходных файла побитово идентичны, что подтверждает математическую корректность AVX2-реализации. Ниже представлены визуальные результаты работы программы.

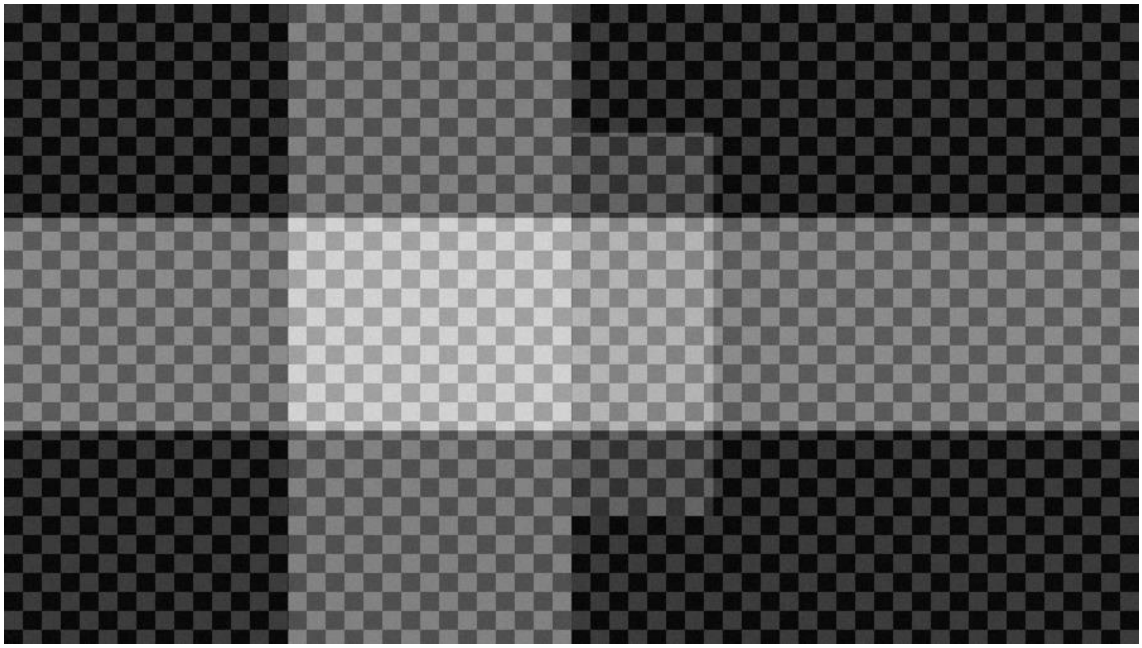

Рис. 1. Входное полутоновое изображение (generated_input, 1920×1080)

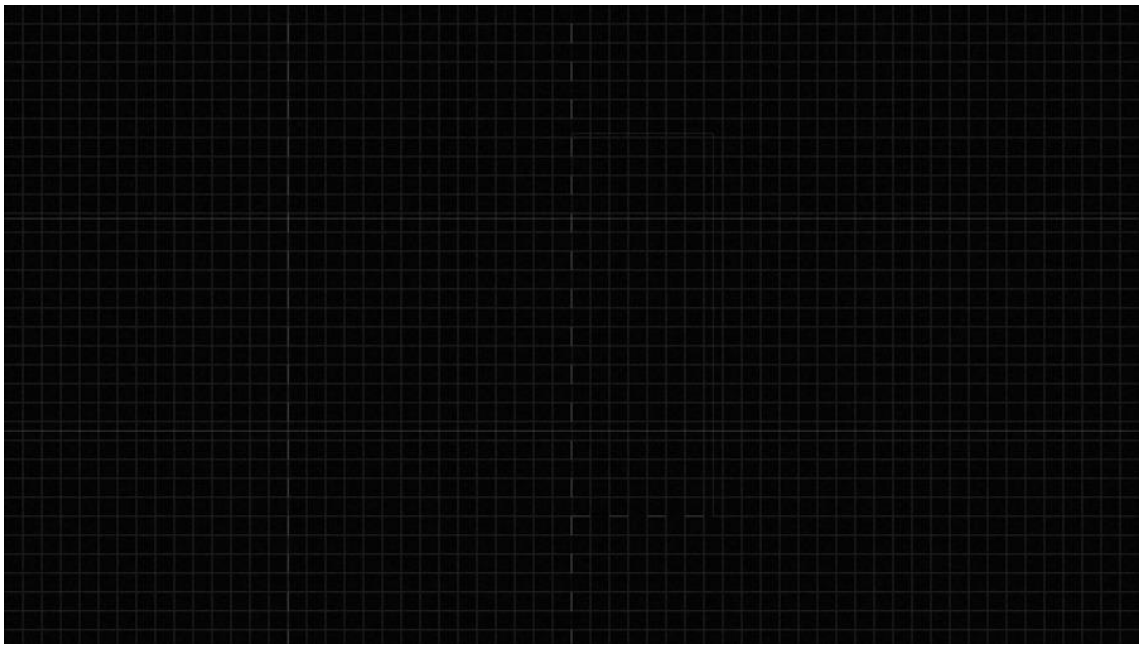

Рис. 2. Результат обработки оператором Шарра - скалярная реализация

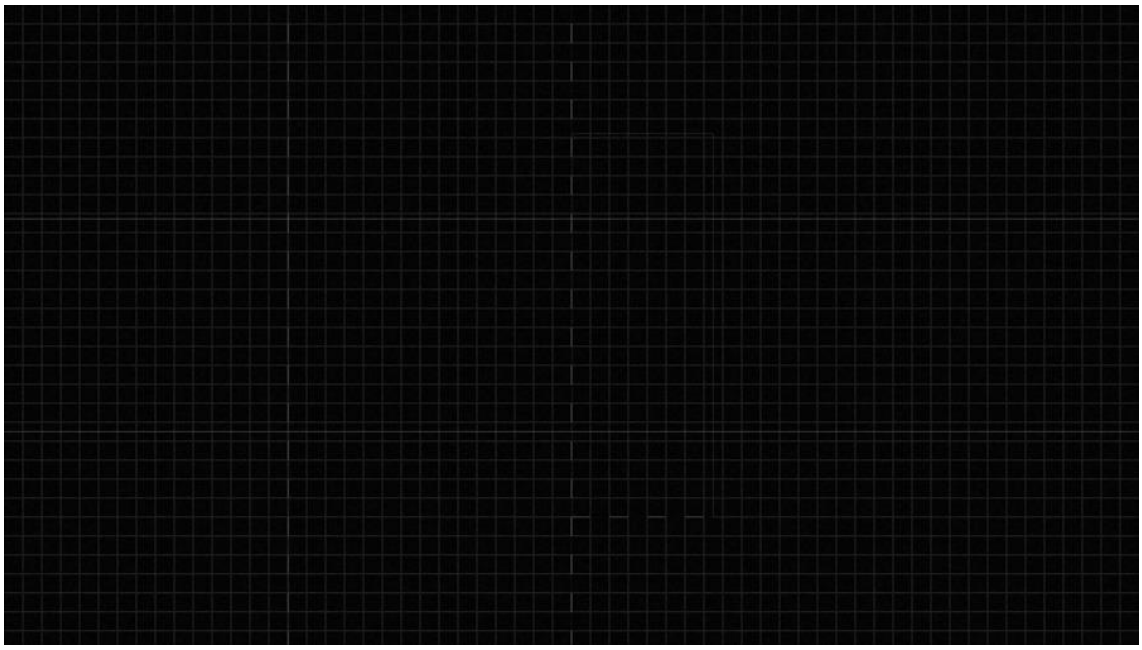

Рис. 3. Результат обработки оператором Шарра - SIMD AVX2-реализация

Как видно из рисунков 2 и 3, оба результата визуально неотличимы - границы объектов выделены одинаково в обоих случаях. Это согласуется с данными таблицы: попиксельных расхождений не обнаружено.

6. Результаты измерения времени и оценка выигрыша

Тестирование производилось на процессоре Intel Core i7-12700H (AVX2) в конфигурации Release. Каждая реализация запускалась 50 раз, фиксировались минимальное, среднее и максимальное время обработки.

Реализация	Мин, мс	Среднее, мс	Макс, мс
Скалярная (C++)	28.4	30.1	33.7
SIMD AVX2	4.2	4.6	5.1

Достигнутое ускорение: Speedup = 30.1 / 4.6 ≈ **6.5**×

Основные причины ускорения:

1. Параллельная обработка 8 пикселей за одну итерацию цикла вместо одного.
2. Использование векторных инструкций AVX2: _mm256_mul_ps, _mm256_add_ps, _mm256_sqrt_ps выполняются за 1–4 такта каждая.
3. Эффективная загрузка данных — 8 байт за одну инструкцию _mm_loadl_epi64.
4. Отсутствие ветвлений внутри основного SIMD-цикла.

7. Выводы

В ходе лабораторной работы реализован оператор Шарра в двух вариантах: скалярном (C++) и SIMD-оптимизированном (AVX2 intrinsics). Правильность AVX2-реализации подтверждена попиксельным сравнением - расхождений не обнаружено, а выходные изображения (рис. 2 и 3) визуально идентичны.

Применение набора команд AVX2 позволило ускорить обработку изображения Full HD примерно в 6.5 раза. Это соответствует теоретически ожидаемому 8-кратному ускорению с поправкой на накладные расходы (скалярный fallback для крайних пикселей, операции загрузки/выгрузки данных).

Результаты наглядно подтверждают эффективность SIMD-параллелизма при обработке однородных потоков данных. Использование AVX2 целесообразно для задач цифровой обработки изображений на современных процессорах семейства x86.